Expert Systems With Applications 227 (2023) 120289

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Per-title and per-segment CRF estimation using DNNs for quality-based video coding

Francisco Micó-Enguídanos *, Wilmer Moina-Rivera, Juan Gutiérrez-Aguado, Miguel Garcia-Pineda

Departament d'Informàtica, Universitat de València, Av. de la Universitat s/n, Burjassot, 46100, València, Spain

ARTICLE INFO

Keywords:
Deep neural network
Video quality
Video coding
HTTP Adaptive Streaming

ABSTRACT

Nowadays, video content accounts for a large percentage of network traffic. Most streaming services use HTTP Adaptive Streaming by splitting the video in non-overlapping segments and by encoding each video segment independently (possibly with multiple representations to allow adaptation to the varying network conditions). In this work we propose an encoding scheme based on a Deep Neural Network (DNN), to perform a per-title and per-segment adaptive estimation of the encoding parameter to achieve a target video quality of the encoded video. A dataset has been prepared using 1212 segments obtained from 158 videos. The segments have been encoded with 19 Constant Rate Factor (CRF) values using the VP9 encoder, generating a total of 23028 encoded segments, and for each encoded video segment, its Video Multi-Method Assessment Fusion (VMAF) quality has been computed. Besides, from a 240p downscaled version of the segments a feature vector has been obtained, and an analysis of the dependency of the features with the resolution has been carried out. With this dataset a DNN has been trained to estimate the CRF to be applied to each segment to achieve a target VMAF quality. Results show that the trained network is able to provide the CRF value to be applied to each video segment to achieve the desired quality with low computational overhead. To validate the proposal, the network has been used to predict the CRF to encode 1840 two-second segments, not used during the training process, using the VP9 codec at Full High Definition (FHD) resolution, and four target quality values. Results show that the system adapts the CRF to each segment and that the final videos have a mean deviation of 1.84% with respect to the requested VMAF value.

1. Introduction

Today, around 62% of the world's population has access to the Internet. This percentage represents 4.95 billions of users, of which 50% use Internet primarily to watch videos, TV shows and movies. They spend more than 3 h per day watching videos, for this reason YouTube is the third most accessed website after Google and Facebook. Moreover, the Internet users spend more than 85$ billion per year on video on demand (VoD) platforms such as *Netflix, Amazon Prime Video, Disney+*, and, others (Kemp, 2022).

Nowadays, most of these streaming services use HTTP Adaptive Streaming (HAS) for over-the-top video distribution. In HAS, a video is divided into smaller chunks called segments and each segment is encoded at several resolutions and bit rates. Each player runs an algorithm to select the most suitable segment in order to create a seamless playout. The goal is to dynamically deliver the highest quality segment to the end user. Although there are many proprietary solutions based on HAS principles, HTTP Live Streaming (HLS) and Dynamic Adaptive Streaming over HTTP (DASH) have achieved commercial success, and have evolved as dominant video streaming formats over the Internet (Begen & Syed, 2018). These streaming systems require efficient video encoding systems to deliver content in as many resolutions, bit rates and codecs as possible to give the best Quality of Experience (QoE) of the end-user (Salvador, Nogueira, & Sargento, 2015).

Today, most video encoders (Richardson, 2002) are based on 6 steps: (a) frame partitioning, (b) predictions, (c) transformations, (d) quantizations, (e) entropy coding, and finally (f) encapsulation (see Fig. 1). The encoder processes a video frame in units of macroblock (step (a)), whose size depends on the codec used. It forms a prediction of the macroblock based on previously coded data, either from the current frame (Intra prediction) or from other frames that have already been encoded (Inter prediction). The encoder subtracts the prediction from the current macroblock to form a residual (step (b)). A block

* Corresponding author.
E-mail addresses: francisco.m.mico@uv.es (F. Micó-Enguídanos), wilmoiri@alumni.uv.es (W. Moina-Rivera), juan.gutierrez@uv.es (J. Gutiérrez-Aguado), miguel.garcia-pineda@uv.es (M. Garcia-Pineda).

https://doi.org/10.1016/j.eswa.2023.120289
Received 27 July 2022; Received in revised form 20 April 2023; Accepted 25 April 2023
Available online 29 April 2023
0957-4174/© 2023 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

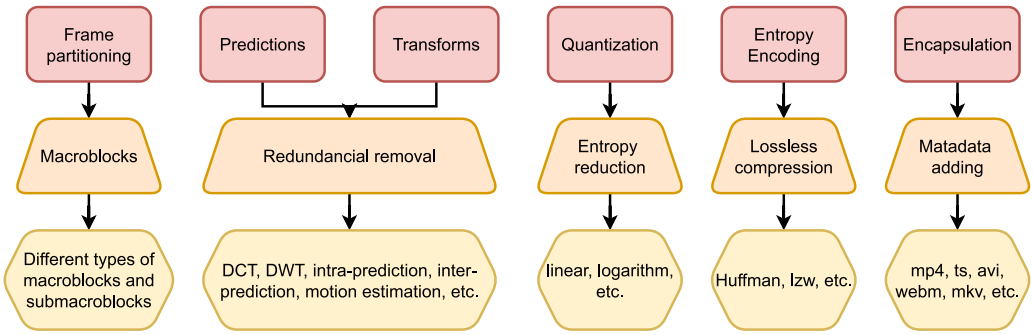


Fig. 1. Main steps of video encoders.

of residual samples is transformed using an integer transform, an approximate form of the Discrete Cosine Transform (DCT). The transform outputs a set of coefficients, each of which is a weighting value for a standard basis pattern (step (c)). The output of the transform, a block of transform coefficients, is quantized, i.e. each coefficient is divided by an integer value. Quantization reduces the precision of the transform coefficients according to a quantization parameter (QP), this is done in step (d). Setting QP to a high value means that more coefficients are set to zero, resulting in high compression at the expense of poor decoded frame quality. Setting QP to a low value means that more non-zero coefficients remain after quantization, resulting in better decoded frame quality but lower compression. The video coding process produces values that must be encoded to form the compressed bitstream. These values and parameters (syntax elements) are converted into binary codes using lossless compression techniques (step (e)). Each of these encoding methods produces an efficient, compact binary representation of the information, which is encapsulated in proper container that allows an encoded bitstream can then be stored and/or transmitted (step (f)).

As we have seen previously, there is a trade-off between quality and compression, and this trade-off is regulated by the low-level QP parameter at the encoder. But this QP value in video coding is linked to a bit rate or CRF value at high level, since each codec can have different QP values. But if we want to optimize the encoding process for a specific resolution and segment, what bit rate or CRF should we select at each moment? Is this selection independent of the content/video type? What is the bit rate or CRF value required to achieve a target perceivable quality? In order to answer these questions, we present a system capable of encoding a video given the desired value of an objective quality metric, such as VMAF (Li, Aaron, Katsavounidis, Moorthy, & Manohara, 2016). Starting from a 4K resolution video, the proposed system downscales the video to low definition (240p) and for each segment various features (such as average luminance, average chrominance, frame rate, etc.) and their statistics (mean, variance, kurtosis, etc.) are extracted. This feature vector and the target VMAF are the inputs of a Deep Neural Network (DNN) in order to give us the CRF to encode the video segment such as the VMAF is close to the target. This proposal has been tested with 1840 two-second video segments from 11 videos, which were not included in the training and validation process, for the following target VMAF values: 80, 85, 90 and 95, thus carrying out 7360 CRF predictions.[1] According to the results, the mean obtained VMAF deviation for a target VMAF = 80 is 2,9%, for a target VMAF = 85 the mean difference is 2,28%, for a target VMAF = 90 this difference is 1354%, and for a target VMAF = 95 the difference decreases to 0,83%.

Therefore, the main contributions of this work are:

- Propose a novel per-title and per-segment coding scheme based on DNN to obtain adaptively the CRF of each segment to attain a target video quality.
- Perform an analysis of the selected features of each video segment with the resolution to select those that show a linear correlation.
- Use the features obtained from the low resolution segments to train the DNN.
- Achieve a prediction time per segment (involving downscaling, feature extraction, CRF prediction using the trained DNN) of less than 4 s.

This paper is organized as follows. Section 2 shows several related work in order to present the main differences between our system and previous proposals. Section 3 describes our proposal including the selected features, an analysis of the dependency of the features with the resolution, details about the generated dataset used in the training phase, and the architecture of the neural network. Section 4 presents the experiments carried out with a dataset of 11 new videos and the results obtained when encoding the videos requesting different quality levels. A brief comparison among related work and our proposal is shown in Section 5. Finally, the conclusions and future work are presented in Section 6.

2. Related work

In recent years, Deep Learning (DL) techniques are being used for detecting situations or estimate data from video content. For example, in Sabokrou, Pourreza, Li, Fathy, and Zhao (2021), the authors propose an effective heart rate estimation method based on DNN from face video. Their proposal works in a proper way in live scenarios and it improves the state of the art in this topic. Another field where there are several works using DL techniques is event/thing detection in video surveillance. Lim, Tang, and Chan (2014) present an intelligent framework for detection of multiple events. Their proposal is based on the principle of compositionality, where surveillance problems are divided into a set of variables comprising regions-of-interest. Through this principle, multiple events can be detected in a surveillance scenario.

Up to our knowledge, there is no work that uses neural networks with the aim of predicting encoding parameters from video features that will ensure the achievement of a desired quality in terms of VMAF quality metric. But recent research papers have presented solutions that use neural networks where the objective is related to the achievement of specific bit rates. These previous works are presented below and the main differences with respect to the proposal presented in this paper are listed.

With the help of a linear model that predicts the bit rate of a video as a function of its resolution, its frame rate, the CRF used for its encoding, and hidden parameters that are video dependent, Covell, Arjovsky, Lin, and Kokaram (2016) have developed a neural network capable of predicting the correct CRF value for a desired bit rate. To predict the CRF, from a first pass transcoding segment dependent features are

[1] A sample demo of a video encoded with two target VMAF quality values using the proposed scheme is available in the following link: https://links.uv.es/jgutierr/dnnVideoCoding.

computed and the model parameters are estimated. With the features of that previous transcoding, they obtained that 80% of the test transcodes were at or below the 20% bit rate error target. Key differences with our work are that we do not impose any model and the network outputs the estimated CRF to achieve a target quality.

Sun, Jin, Li, and Li (2018) also propose a regression neural network to estimate parameters of a second-degree model. Segment-dependent model parameters are obtained using features computed from a first-pass transcoding, then the CRF is obtained to achieve a target bit rate. The key difference with respect to Covell et al. (2016) consists in using a second degree model that improves a 8% the previous results, but the differences with respect to our proposal are still the aforementioned ones.

Cheng and Zhang (2019) propose a single-pass coding by adding a lookahead module that implements a scene-cut detector to divide the input video into different GOPs and performs intra and inter coding on subsampled frames for an estimation of the encoding cost. Thus, with the help of the information generated by the lookahead module for each GOP, a neural network predicts the CRF to achieve a given bit rate. According to the authors of this paper, the results obtained improve those of Covell et al. (2016) but not those of Sun et al. (2018). Although this framework is more efficient thanks to single-pass encoding, it is still focused on bit rate limitation and therefore cannot ensure the same quality for the entire video.

FAUST (*Fast Per-Scene Encoding Using Entropy-Based Scene Detection and Machine Learning*) is presented by Zabrovskiy, Agrawal, Timmerer, and Prodan (2021) and its main contributions are a fast video scene detection at low resolution and low bit rate to split videos scenes, a neural network for predicting the quality metric of video segments and a fast per-scene encoding that does not require multiple test encodings with the aim of achieving an optimized bit rate ladder construction. There are many points that differentiate this work from the one presented in this paper. The first one would be the features taken as input to the neural network. In their proposal they do not take into account video features such as luminance or chrominance values, hue, saturation, entropy, etc. As quality metric they use the luminance Peak Signal-to-Noise Ratio (Y-PSNR). This metric is one of the objective metrics that has the worst relationship with the Mean Opinion Score (MOS). Nowadays the VMAF metric has a better correlation with the subjective quality perceived by an end user than other objective quality metrics, like PSNR or the Structural Similarity Index Measure (SSIM) (Nandakumar, Wu, Wei, & Ten-Ami, 2019).

As stated by Moina-Rivera, Gutiérrez-Aguado, and Garcia-Pineda (2021), efficiency of the coding process (quality vs size and time) must be maximized and they proposed an adaptive quality based encoding scheme: for each video segment of a raw 4K input video a CRF was iteratively obtained to achieve a target VMAF quality. This scheme showed a significant improvement with respect to the case in which a constant CRF was selected. In Moina-Rivera, Gutiérrez-Aguado, Garcia-Pineda, and Claver (2021) it is shown that aggressive downscaling of segments can help greatly in increasing the efficiency of the coding process. Following these ideas, we propose a scheme in which we include a neural network capable of predicting from extracted features of downscaled segments the needed CRF to achieve a target VMAF.

To conclude this section, we are able to affirm that there is not any paper where the authors present a system capable of offering a CRF value based on some visual features of a video, like luminance, chrominance, hue, saturation, etc., to guarantee a certain objective video quality.

3. Proposed framework

In this section our proposal will be described, as well as the feature extraction and the dataset for training and validation of the same proposal (see Fig. 2).

Table 1
2s-chunk splitting and feature extraction times (s) per resolution.

Video name	Chunk splitting /Downscaling		Feature extraction	
	Mean	Std. dev.	Mean	Std. dev.
Sintel	3.799	0.289	0.736	0.321
Tears of Steel	3.635	0.781	0.772	0.334
Eldorado	1.926	0.274	0.852	0.338
Indoor Soccer	2.052	0.834	0.829	0.216

(a) 240p resolution

Video name	Chunk splitting		Feature extraction	
	Mean	Std. dev.	Mean	Std. dev.
Sintel	7.135	3.544	31.403	22.034
Tears of Steel	8.862	6.299	32.678	22.622
Eldorado	6.576	4.470	36.396	24.474
Indoor Soccer	6.107	4.231	36.240	24.609

(b) 2160p resolution

The main reason why the feature extraction is performed on a downscaled video is the reduction of computation time. Table 1 shows the computational time savings when using 240p 2s-segments versus 2160p 2s-segments to compute the features.[2] The creation of the 240p video segments, when the video is in 2160p, involves splitting and downscaling processes while to create the 2160p video segments it is only required the splitting process. As shown is the Table 1, the time used to create the segments in 240p is about 3 times faster than in 2160p. Results for features extraction are even more drastic: 40 times faster in 240p resolution than in 2160p resolution.

In the following subsections, the computed features for each segment will be described, and the dependency of the features with the video resolution will be analyzed. The description of the dataset and details about the DNN architecture will be also provided.

3.1. Feature extraction

As video codecs exploit spatial and temporal redundance, we have selected initially 42 features related to spatial information, color information, and temporal information. Using the ffprobe tool with *signalstats* and *entropy* filters (command 1 in Table 2) and using the Python *siti* v1.4.5 package (Robitza & Krasula, 2022) according to ITU-T (ITU-T, 1999) (command 2 in Table 2), we have obtained the set of features in Table 3.

With the Python *numpy* v1.8.1 (Harris et al., 2020) and *scipy* v1.22.4 (Virtanen et al., 2020) packages, we have calculated for each segment the mean, the standard deviation, the kurtosis, and the skewness for each of the aforementioned features. Finally, video scene detection (with *scdet* filter of the ffprobe tool - command 2 in Table 2) and frame rate were taken into consideration. In total, 42 features per video segment were computed.

3.2. Dependency of the features with resolution

In this section we analyze the dependency of the features with respect to the resolution of the video segments. We have shown the time savings of computing the features from a downscaled version of the video segments (see Table 1 at the beginning of this section). But, we need to verify if the computation of the features can be performed in a low resolution version of the video segments without losing information.

[2] Videos used in this test are described in Section 4 Feature extraction is explained in next Section 3.1.

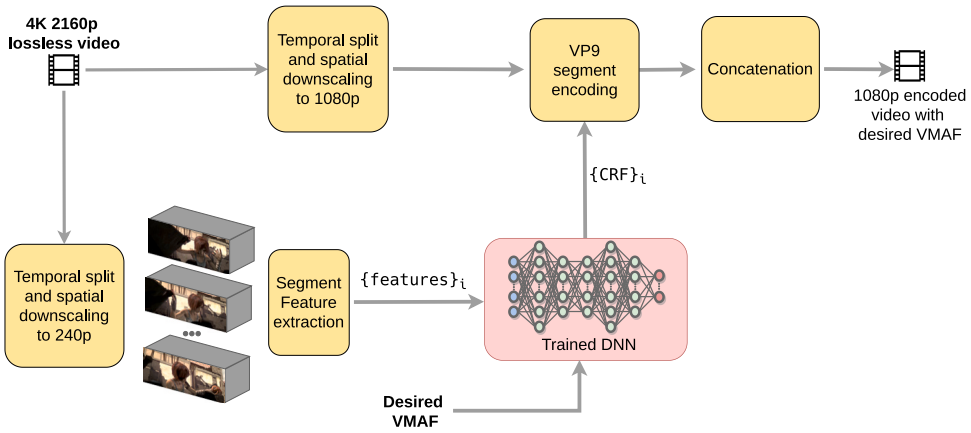


Fig. 2. Proposed coding scheme: the CRF for each segment is estimated using a DNN and each segment is encoded at 1080p resolution using the estimated CRF.

Table 2
Commands for feature extraction.

Commands:	
1	`ffprobe -f lavfi -i ''movie=video_segment.mp4,entropy,scdet,signalstats'' -show_frames -show_streams -select_streams v -of json`
2	`siti -of json -q -f video_segment.mp4`

Table 3
Set of features obtained with the ffprobe tool and Python siti package.

Features per frame obtained with the ffprobe tool	
Y, U, V	luminance (Y) and chrominance values (U, V)
SAT	saturation value
HUE	value for hue
$S(Y)$, $S(U)$, $S(V)$	normalized graylevel entropy in histogram of luminance and chrominance channels
Features per frame obtained with the Python siti package	
SI	spatial information
TI	temporal information

For this analysis we have used only the segments obtained with the videos from the UVG dataset (Mercat, Viitanen, & Vanne, 2020) (a total of 47 2s-segments). We have converted the original segments at 2160p resolution to the resolutions 240p, 480p, 720p, 1080p and 1440p, and for each of the resolutions we have analyzed the mean values of the following features: Y, U, V, SAT, HUE, $S(Y)$, $S(U)$, $S(V)$, SI, and TI.

In Fig. 3 we show the values obtained for each feature with lower resolutions versus the same values obtained with the original resolution.

In some cases we can observe a linear behavior between the feature and the resolution. This is the case for Y, U and V (Figs. 3c, 3e and 3g), and SAT (Fig. 3i) features. This fidelity between the data obtained at low resolutions and those obtained at high resolutions confirms that the lower resolution can be used without losing of information.

In the other cases we see that the change of resolution has some impact on the features. In the case of the normalized entropy of Y, U and V (Figs. 3d, 3f and 3h), TI (Fig. 3b) and HUE (Fig. 3j) we can see that there are differences, being more pronounced at lower resolutions. But in any of these cases, it can be observed that whatever the resolution used, near-linear behavior is still observed with respect to the data obtained with the original resolution with little deviations, although not very pronounced. This near-linearity between the features obtained at high resolution and those obtained at low resolution allows us to take advantage of the features computed at 240p because they continue to characterize the segments with sufficient fidelity.

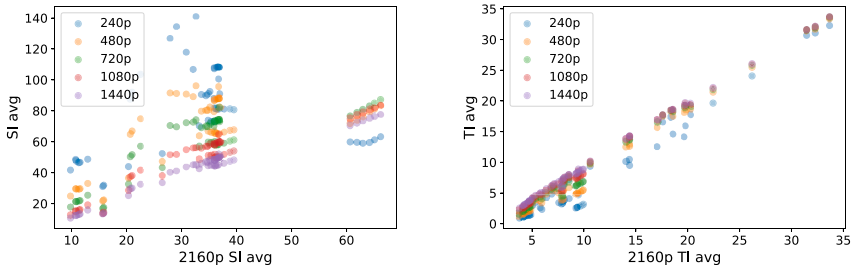
In the case of the SI feature (Fig. 3a) we find large deviations and total loss of proportionality between the data obtained at 2160p and those obtained at lower resolutions. In the case of the points corresponding to the data obtained at the lowest resolution (240p), which is the one we are interested in, it can be seen that there is no possibility of obtaining a consistent description to explain the distribution of these points.

Consequently, this feature and its statistics (four features) have been discarded because their values obtained with the 240p resolution do not correlate with those obtained for other resolutions. Therefore, a total of 38 features, instead of the 42 originally considered, were finally taken into consideration.

3.3. Dataset description

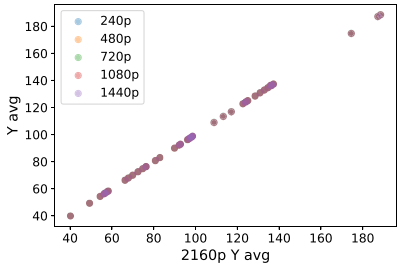
For the training of the neural network, we have built a dataset from the following 4K videos: the MCML 4 K UHD video quality database (Cheon & Lee, 2018), the open Ultra Video Group (UVG) dataset (Mercat et al., 2020), the IVC UHD videos dataset (Vigier, Rousseau, Da Silva, & Le Callet, 2016), the SJTU 4 K video sequence dataset (Song, Tang, Zhang, Yang, & Xia, 2013), the AVT-VQDB-UHD-1 video quality database (Rao, Göring, Robitza, Feiten, & Raake, 2019) and the YouTube User Generated Content (UGC) dataset (Wang, Inguva, & Adsumilli, 2019). Discarding the repeated sequences, a total of 1212 two-second video segments from 158 4K (3840 × 2160) videos were finally considered.

From the videos found in the datasets used, we can indicate that the essential parameter range is quite large including high variations in contrast, motion and texture:

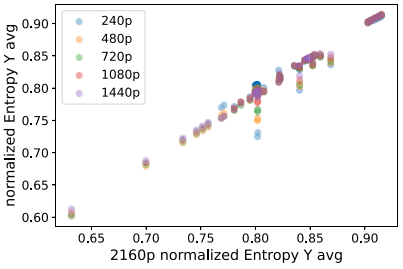


(a) SI

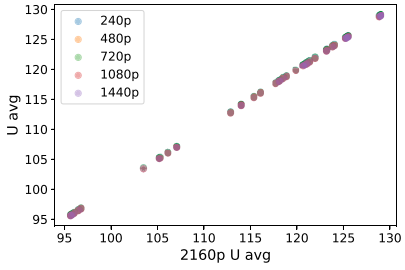
(b) TI



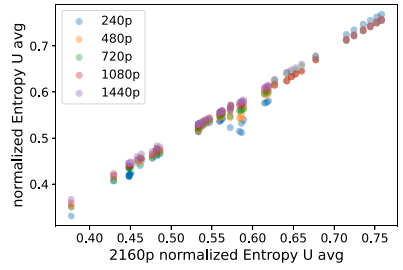
(c) Y



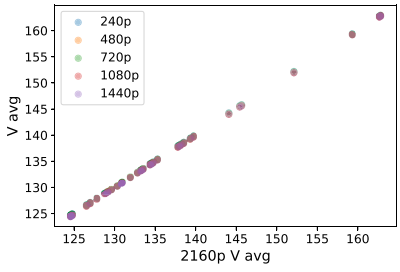
(d) Normalized entropy of Y



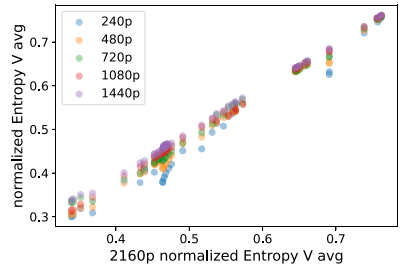
(e) U



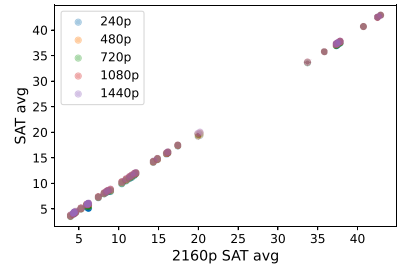
(f) Normalized entropy of U



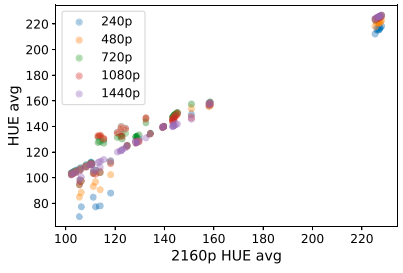
(g) V



(h) Normalized entropy of V



(i) SAT



(j) HUE

Fig. 3. 2160p metrics vs 240p, 480p, 720p and 1080p metrics.

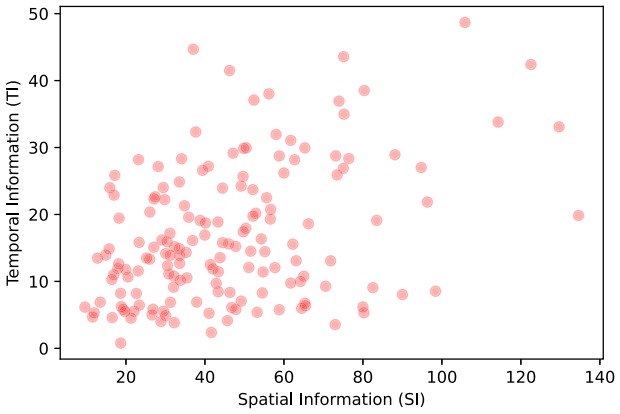


Fig. 4. SI and TI average scores of the selected video sequences.

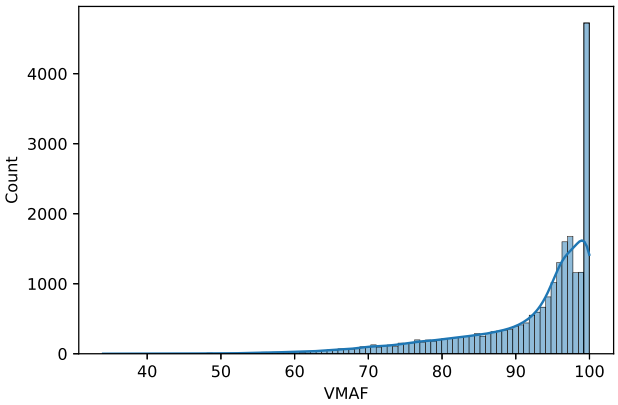


Fig. 5. VMAF histogram from training data.

- Frame rates: 23.98 fps (3.8%), 24 fps (0.6%), 25 fps (25.3%), 29.97 fps (22.8%), 30 fps (19.6%), 50 fps (5.7%), 59.94 fps (7.0%), 60 fps (10.1%) and 120 fps (5.1%).
- Duration: 2.5s (0.6%), 5s (4.4%), 6s (0.6%), 8s (3.8%), 9s (0.6%), 9.9s (0.6%), 10 s (9.5%), 12 s (21.5%), 19.9s (23.4%) and 20 s (34.8%)
- Bit depth: 8 bits (100%)
- Chroma sampling: 4:2:0 (100%)
- Containers: mkv (63.9%), yuv (31.6%) or y4 m (4.4%)

The spatial information (SI) and temporal information (TI) scores of the selected video sequences are distributed across a wide range of SI and TI values as shown in Fig. 4.

From the selected videos, we have created lossless non-overlapping 2-second segments in 2 resolutions (426 × 240 and 1920 × 1080) with the ffmpeg tool (FFmpeg Team, 2022), always keeping the aspect ratio. Each segment has been encoded with the H264 encoder with the Constant Rate Factor control mode (CRF=0). Ultrafast preset and 8-bit YUV 4:2:0 color sampling were used and CABAC (Context Adaptative Binary Arithmetic Coder) was not enabled. Bicubic algorithm was applied to perform the downscaling process. Thus, for each resolution we obtained 1212 segments.

The 240p segments have been used to extract the 38 selected representative features from the video segments that will serve as input for the neural network as described in Section 3.2.

The 1080p segments were compressed with VP9 using ffmpeg with libvpx support (Cock, Mavlankar, Moorthy, & Aaron, 2016) (command 1 in Table 4). We have selected VP9 because it is an open and royalty-free video coding format, which has an improvement around 30-40% over x264 (Cock et al., 2016) besides a huge web compatibility (more than 94% of users could play these videos on a web browser).[3]

Each 1080p segment was encoded using 19 CRF values: {12, 15, 17, 19, 21, 23, 25, 27, 29, 31, 34, 37, 40, 43, 46, 49, 52, 55, 58} obtaining a total of 23028 coded segments. The VMAF metric (Li et al., 2018) was computed for each encoded segment with the ffmpeg+libvmaf v2.3.0 tool (command 2 in Table 4).

Therefore, for each 4 K segment we have the following data: its 38 features obtained from the 240p resolution, the VMAF, and the CRF that yields that VMAF (when encoding the video using VP9 at 1080p resolution).

3.4. Neural network

Data must be processed before being fed into the neural network. As described in Section 3.2, from the dataset we have selected 38 features for each video segment. In addition from the coded segment we also have the CRF value used in each case and the corresponding VMAF metric.

An analysis of the VMAF values obtained from the training dataset reveals a pronounced peak between values 99 and 100 which could put a strain on the learning process of the neural network. The histogram of the VMAF values of the original training set is shown in Fig. 5. As encoded videos with VMAF values larger or equal to 95 are almost indistinguishable from the original video signals (Kah et al., 2021; Rassool, 2017), we decided to limit the VMAF scores of the training data to values between 0 and 99.

Almost all numerical values have been normalized using standardization (transformation that sets the mean of data to 0 and the standard deviation to 1); scene detection has been set as a binary value and frame rates have been vectorized using one-hot encoding in 9 categories.

Therefore, the training data is formed by: extracted features with their standard deviation, their kurtosis, and their skewness along with scene detection (in total 38 values), frame rate categories (9), and VMAF. The output is a 19 multiclass classification that will provide the CRF that must be applied to the video segment to achieve the desired VMAF.

For the implementation of our neural network we have used the Python packages KerasTuner v1.1.0 (O'Malley et al., 2019), Keras v2.7.0 (Chollet et al., 2015) and TensorFlow v2.7.0 (Abadi et al., 2015).

The neural network we present is a 5-layer densely connected sequential model with 100, 68, 50, 100, and 50 nodes respectively with rectified linear unit (ReLU) activation function. The inputs are represented with a layer with 47 nodes and the model is ended with a layer of size 19 with softmax activation function, where each node encodes a different CRF output class (see Fig. 6).

The loss function used during training is the Categorical Cross-Entropy that computes the cross-entropy loss between the labels and the predictions. Using the Categorical Cross-Entropy loss we train the DNN to output a probability distribution over the 19 outputs (one output for each CRF value). Therefore, the DNN is used for multi-class classification: given an input we will choose the class (in our case the class is a specific CRF value) that gives the highest probability. The optimizer selected for this model is the Adam algorithm, a stochastic gradient descent method based on adaptive estimation of first-order and second-order moments. And finally, the measure of success that will be monitored during the training is the accuracy computed as the number of right classified outputs divided by the total number of inputs, where values close to one represents a better trained model.

With the aim of avoiding overfitting, the EarlyStopping callback function from Keras was used to stop the training when monitored loss revealed that no improvement was achieved after 10 epochs. It is noted here that *restore_best_weights* argument of the callback function was set

[3] https://caniuse.com/?search=webm

Table 4
Sample commands for VP9 encoding, for a video with 24 fps, and VMAF computation.

Commands:	
1	`ffmpeg -i video_segment_1080.mp4 -c:v libvpx-vp9 -crf crf -force_key_frames ''expr:gte(t,n_forced*2)'' -g 48 -b:v 0 -speed 2 -quality good -row-mt 1 -threads 24 video_segment_1080_codVP9.mp4`
2	`ffmpeg -i video_segment_1080_codVP9.mp4 -i video_segment_1080.mp4 -lavfi libvmaf=''n_threads=24:log_fmt=json:log_path=vmafresult.json'' -f null -`

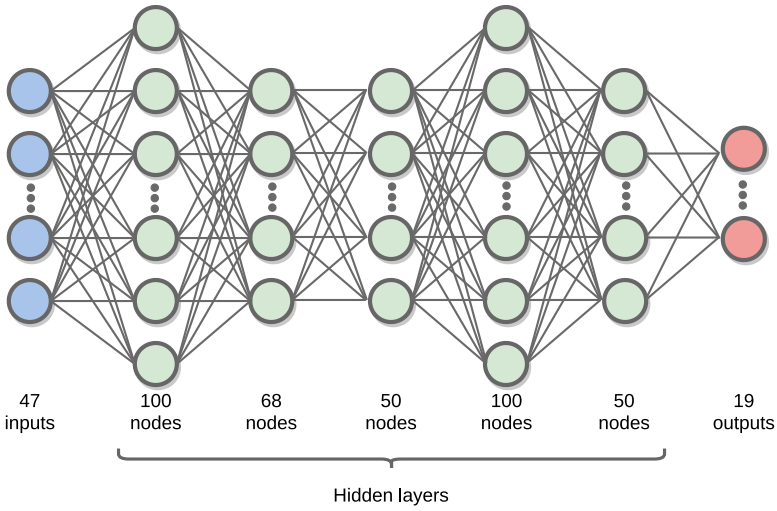
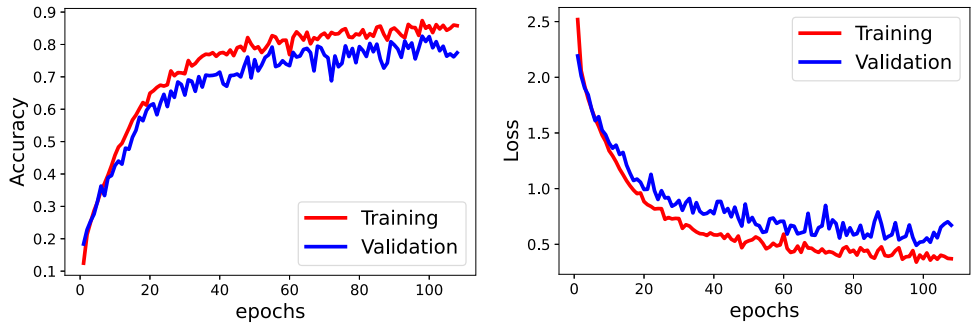


Fig. 6. Neural network structure.



(a) Training and validation accuracy (b) Training and validation loss

Fig. 7. Accuracy and loss curves.

to true with the aim of restoring model weights from the epoch with the best value of monitored loss.

Figs. 7(a) and 7(b) show loss and accuracy curves that were obtained during training. It can be appreciated that after approximately 100 epochs the accuracy for training is near 90% and above 80% for validation. Losses for both training and validation reach minima below 0.5.

4. Experiments and results

In this section, we present our testbed, the experiments performed and their results in order to validate our proposal.

4.1. Description of the testbed and videos

The experiments were performed in a virtual machine (VM) running in a server with 2 CPUs Intel(R) Xeon(R) silver 4210 @2.20 GHz (both with 10 cores and 20 threads and a L1-cache size of 640 KiB, a L2-cache size of 10 MiB and a L3-cache size of 13 MiB) and 128 GiB DIMM DDR4 2666 MHz RAM. The server operating system is Xubuntu 20.04.4 LTS (GNU/Linux 5.4.0-121-generic x86_64) with KVM hypervisor. The VM has 24 vCPUs and 48 GB RAM. All the videos were stored in a 8 TB ST8000DM004-2CX1 5400 RPM disk attached to the VM using PCI-passthrough.

In order to evaluate and validate the proposed method, several tests have been carried out over several 4 K videos that were not used during the training phase of the DNN. Details of these videos are shown in Table 5. The first nine videos are from CableLabs.[4] and, in an attempt to bring the proposal closer to real environments, we have also selected two videos with a duration longer than 10 minutes, which are SIN[5] and

[4] https://www.cablelabs.com/4k/
[5] https://durian.blender.org

Table 5
Videos used in the experiments.

Video	Acronym	Frame rate	Duration	Size (W × H)
Ancient Thought	ANC	23.98 fps	3 min 07 s	3840 × 2160
Eldorado	ELD	23.98 fps	3 min 02 s	3840 × 2160
Indoor Soccer	IND	23.98 fps	4 min 11 s	3840 × 2160
Life Untouched	LIU	59.94 fps	3 min 17 s	3840 × 2160
Lifting Off	LIO	23.98 fps	3 min 21 s	3840 × 2160
Moment of Intensity	MOM	59.94 fps	3 min 10 s	3840 × 2160
Seconds That Count	SEC	23.98 fps	5 min 22 s	3840 × 2160
Skateboarding	SKA	23.98 fps	4 min 35 s	3840 × 2160
Unspoken Friend	UNS	23.98 fps	4 min 17 s	3840 × 2160
Sintel	SIN	24 fps	14 min 48 s	4096 × 1744
Tears Of Steel	TOS	24 fps	12 min 14 s	4096 × 1714

TOS[6] In addition, SIN and TOS do not have a 16:9 aspect ratio, which will allow us to check if our system adapts to other types of aspect ratios such as 2.35:1.

4.2. Per-title and per-segment CRF adaptation

Following the procedure described in Section 3, after creating the 2s segments, downscaling, and extracting the features for each of them, predictions about the CRFs to use for segments encoding at FHD (Full High Definition) resolution were made for the following target VMAF values: 80, 85, 90 and 95.

A important aspect to take into account when making predictions is the time needed to achieve them. It is important to consider whether the predictions will be made sequentially segment by segment (as in the case of live video streaming) or in batches (as in the case of video on demand streaming).

In the case of making the prediction for a single segment at a time, in our tests we obtained average prediction times of 69.8 ms. In the case of batch predictions, we have seen that videos with more segments were more advantageous. In the case of the CableLabs videos (with around 100 chunks each) the average per chunk was down to 0.76 ms, while with the Sintel and Tears Of Steel videos, with around 400 chunks each, the average was down to 0.24 ms.

Fig. 8 shows boxplots with the distribution of the CRF values predicted by the neural network for a subset of the videos. As expected, the higher target VMAF, the lower CRF values used. Overall, we can also note that there is a greater variability for higher target VMAFs.

If we pay attention to the CRFs obtained for the VMAF target equal to 80 (see Fig. 8(a)), we observe that in the case of ELD 50% of the values are in the range of 46 to 52, while for IND the range goes from 49 to 55. LIU is the video that provides us with the lowest CRFs, ranging from 43 to 49, while for the rest of the videos the range goes from 52 to 58. As can be seen, for most of the videos we obtain high CRFs. This indicates that we are asking for a rather low VMAF quality for the output resolution.

By increasing the target VMAF to 85 (see Fig. 8(b)), the CRFs values, obtained for each segment, decrease as a whole to improve the quality of the resulting video. 50% of the values are in the range of 40 to 49 in the case of ELD, for IND this same range changes and goes from 43 to 52. For LIU 50% of the values vary between the range of 37 to 43, while for SKA video this range goes from 56 to 42. In the case of SIN and TOS, 50% of the CRFs obtained by our proposal range from 46 to 55. As with the VMAF target of 80, the video with the lowest CRF range is LIU, while the highest CRF ranges are for SIN and TOS.

When we ask our system for a quality equal to 90 in VMAF, the results are shown in Fig. 8(c). 50% of the values for ELD are between 34 and 43. In the case of IND video that same range goes from 37 to 46, for LIU that range changes from 31 to 37. With the SKA video 50% of the values obtained by our proposal are between 40 and 46, while for SIN and TOS this range goes from 40 to 49. For this quality value, 50% of the values fall within a maximum range of 9 CRF values, as was the case with a VMAF target of 85.

Fig. 8(d) shows the variability of CRFs obtained by our proposal when we require a VMAF = 95. In this case, being the highest quality request, the proposal gives us the lowest CRF values of the experiments. For ELD and IND, the range of CRFs from 25 to 34 represents 50% of the CRFs obtained, for LIU this range changes and goes from 15 to 23, being this video the one with the lowest CRFs. For SKA 50% of the values range from 27 to 34, while for SIN this range goes from 29 to 40 and for TOS from 31 to 40.

In general, we can indicate that as we increase the target quality, more outliers appear. The SKA, SIN and TOS videos, which have a higher range of CRFs, have the lowest mean Y and the highest U and V values. Another relationship that can be observed is the connection of the mean value of CRF and its rank with the TI parameter. In this case, the LIU video has the lowest mean TI value (2.83) and requires a lower CRF value. SIN and TOS are the videos with the highest mean TI, 15.21 and 16.83 respectively. These videos require the higher CRF values and the higher rank. These relations are observed in all VMAFs analyzed.

4.3. Quality-driven video coding results

Table 6 shows the average VMAF obtained among all the segments of each video with respect to target VMAF value. As a general comment of these results, the mean obtained VMAF deviation for a target VMAF of 80 is 2,9%, for a target VMAF of 85 the mean difference is 2,28%, for a target VMAF of 90 this difference is 1,35%, and for a target VMAF of 95 the difference decreases to 0,83%. This can be due to the fact that we have more training data in the VMAF range $[90, 95]$ as shown in Fig. 5.

Kah et al. (2021) have demonstrated that the maximum difference of two VMAF scores, for which the associated subjective qualities are approximately the same on average, is determined to be 2. Table 6 shows the differences between average VMAF obtained and target VMAF for each video.

It is worth noting that better results are obtained with higher target VMAF values. In particular for the target VMAF value 95 all the differences are below 2 (with a minimum of 0.226 and a maximum value of 1.522) and, consequently, it could be argued that most of the segments have been encoded in such a way that they are indistinguishable from a segment for which we would get a VMAF of 95. For the target VMAF value of 90, 82% of the differences are below 2 (with a minimum of 0.018 and a maximum of 1.786); the rest is superior than 2 but inferior than 3 (with a maximum of 2.985). For the target VMAF values of 80 and 85, 55% of the differences are below 2, 18% between 2 and 3 and 27% above 3 (with a very high maximum of 6.285).

Per video, 6 of them (ELD, IND, LIU, SKA, SIN and TOS) have obtained the best results with no differences above 2. The ANC video has obtained the worst results with greats differences respect to 80 and 85 target VMAFs. This video is a very dark video with slow motion where practically all its scenes include a high number of black pixels. Such errors may be mainly due to the fact that our training dataset does not include many videos with these characteristics.

Figs. 9 and 10 show the results at visual level of the proposed quality-based encoding system for the ELD and IND videos. These figures show a frame, and for a given region we zoom in to visualize the differences when it is encoded with a given target VMAF.

Fig. 9 show a frame of ELD video. Fig. 9(a) shows a lossless frame with a red square, which delimits the region where we are going to pay attention in the adjacent figures to analyze the quality according to the VMAF value. The zoom of this area is shown in Fig. 9(b). In Figs. 9(c)–9(f) we can see the CRF applied in each case and real value

[6] https://mango.blender.org

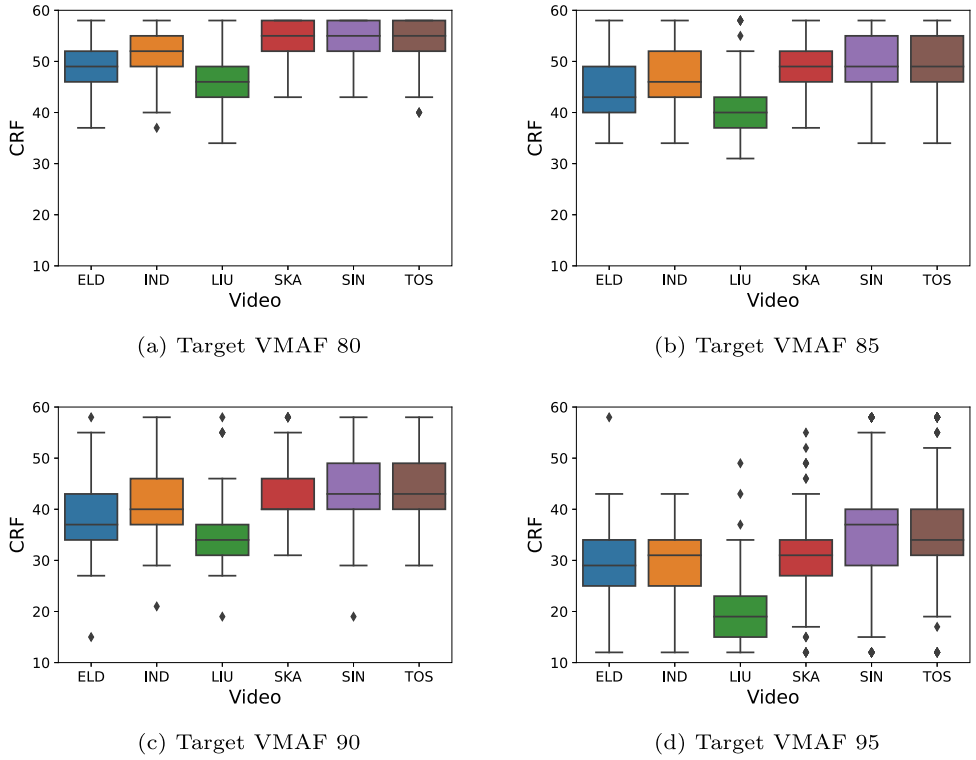


Fig. 8. Predicted CRF per video for each target VMAF.

Table 6
Average VMAF obtained and differences respect to target VMAF.

Video	Target VMAF							
	80		85		90		95	
	Avg.	Diff.	Avg.	Diff.	Avg.	Diff.	Avg.	Diff.
ANC	86.285	6.285	88.828	3.828	91.688	1.688	95.367	0.367
ELD	79.05	−0.95	84.336	−0.664	89.982	−0.018	95.242	0.242
IND	81.137	1.137	85.482	0.482	90.677	0.677	96.224	1.224
LIU	79.648	−0.352	84.776	−0.224	89.716	−0.284	94.135	−0.865
LIO	77.142	−2.858	80.277	−4.723	87.015	−2.985	93.742	−1.258
MOM	83.792	3.792	87.845	2.845	91.786	1.786	96.333	1.333
SEC	84.556	4.556	88.02	3.02	92.328	2.328	96.522	1.522
SKA	79.324	−0.676	85.169	0.169	90.846	0.846	95.715	0.715
UNS	77.45	−2.55	82.814	−2.186	89.236	−0.764	95.627	0.627
SIN	79.168	−0.832	83.747	−1.253	89.025	−0.975	94.69	−0.31
TOS	78.346	−1.654	83.327	−1.673	88.946	−1.054	94.774	−0.226

of VMAF, which in some cases has a small deviation from the target VMAF.

In Fig. 9(c) we see how using a CRF value of 49 permits to obtain a value close to 80 in VMAF, it causes losses of information such as some branches and water drops that are observed in Fig. 9(b). These details are perceived from less to more, as a higher VMAF is requested (see Figs. 9(d) and 9(e)). Finally, when our system encodes the video to obtain a VMAF of 95, it is observed that Figs. 9(b) and 9(f) are practically the same. There are few small differences but these differences are imperceptible to the human eye when the image is scaled to 1080p resolution.

Fig. 10(a) shows a frame of IND video, specifically, a frame associated with the second 203 of the video. In this figure there is a red square, which delimits the area to be studied in the rest of the images that make up Fig. 10. Fig. 10(b) is a zoom in of the red area with lossless encoding. The rest of the figures represent the visual quality of the selected area according to the VMAF target. Fig. 10(c) shows the visual quality when a CRF of 52 is used for encoding in order to obtain a VMAF near to 80. In this figure we can see that there are many deficiencies in the image, such as the faces of the people in the audience as well as the profile of the soccer player's arm.

These deficiencies disappear as we increase the VMAF target and therefore decrease the value of CRF required for coding (see Figs. 10(d) and 10(e)). For a VMAF of 95, the human eye has no ability to differentiate between encoded and lossless video. This can be seen in Fig. 10(f), where a CRF of 29 is applied to obtain a VMAF of 94.933. If we compare Fig. 10(f) with Fig. 10(b), we can see that they are practically the same, with a small difference in the tone of the colors. This difference is not detectable when we look at the whole 1080p resolution frame.

5. Qualitative comparison

In this section, we will conduct a qualitative comparison of the papers presented in Section 2 and our proposal. As mentioned in Section 2, to the best of our knowledge, there are no previous works that utilize video characteristics to determine encoding parameters that would ensure a specific video quality. This leads us to make a qualitative comparison, rather than a quantitative one. However, we



(a) Image from ELD video (t=87s)



(b) Cropped image



(c) Cropped image (target VMAF 80)
CRF applied 49 - chunk VMAF 78.406



(d) Cropped image (target VMAF 85)
CRF applied 43 - chunk VMAF 84.505



(e) Cropped image (target VMAF 90)
CRF applied 37 - chunk VMAF 89.726



(f) Cropped image (target VMAF 95)
CRF applied 25 - chunk VMAF 95.753

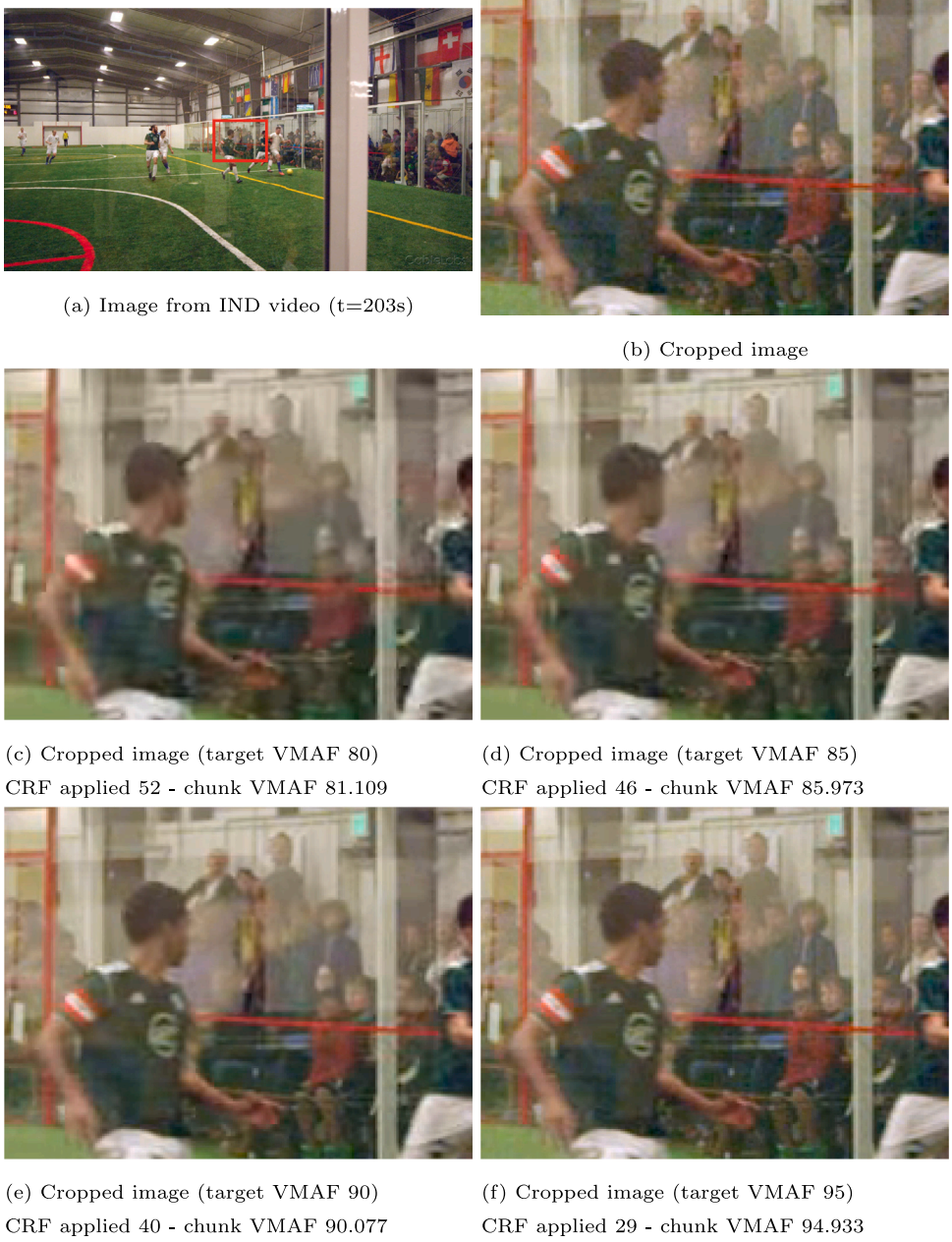
Fig. 9. Coding results for ELD video.

will include some quantitative data such as: training samples, test samples, error rates, etc.

Table 7 compares nine aspects among five previous works and our proposal. The first aspect analyzes which works aim to encode the video to obtain a specific quality. In this case, only the work of Moina-Rivera, Gutiérrez-Aguado, Garcia-Pineda, and Claver (2021) propose a quality-based video encoding system using an iterative algorithm to select a proper CRF for the desired quality. However, the main difference between that work and our proposal is the use of machine learning techniques to achieve a target quality from video features.

The rest of the papers use machine learning techniques to encode videos for purposes other than obtaining videos for a target quality. As shown in Table 7, artificial neural networks are the primary machine learning technique used in previous works. Some of them use shallow neural networks and other ones utilize deep neural network, like our proposal. Our approach is in third place with respect to the number of training and validation samples, as well as for testing samples. But, it is very close to the number of samples in the paper written by Sun et al. (2018), which is in second place.

Concerning the number of video features used as inputs to machine learning systems, we can see that most of them utilize around 8–10 parameters, except for Cheng and Zhang (2019). In our proposal we use 8 features and their basic statistics to obtain a more complete view of each feature throughout each video segment. In addition, it should be noted that the works of Covell et al. (2016), Cheng and Zhang (2019), Sun et al. (2018) require a previous coding process to extract some of the input parameters, while in Zabrovskiy et al. (2021) and our proposal only requires the downscaling of the video segments. This means less coding time and therefore less resource usage, thus improving system performance. Another aspect to highlight is the use of low resolutions to extract video features. This process is partially used (3 of 10 features) by Zabrovskiy et al. (2021). Instead, our system



(a) Image from IND video (t=203s)

(b) Cropped image

(c) Cropped image (target VMAF 80)
CRF applied 52 - chunk VMAF 81.109

(d) Cropped image (target VMAF 85)
CRF applied 46 - chunk VMAF 85.973

(e) Cropped image (target VMAF 90)
CRF applied 40 - chunk VMAF 90.077

(f) Cropped image (target VMAF 95)
CRF applied 29 - chunk VMAF 94.933

Fig. 10. Coding results for IND video.

extracts all the input features from low resolution videos. Table 1 shows the benefits, in terms of time, to extract the video features in low resolution against high resolution.

Lastly, considering the error rates of previous works compared to our proposal, Covell et al. (2016) and Cheng and Zhang (2019) report an error rate of 20% or less in their tests. This rate is relatively high, as it means that 1 out of 5 predictions will be incorrect. In Sun et al. (2018), the error rate ranges from 10% to 20%, which is an improvement over previous papers, but still relatively high for some cases (when it is around 20%). Moina-Rivera, Gutiérrez-Aguado, Garcia-Pineda, and Claver (2021) present an iterative system that provides a CRF for a target quality (VMAF) with a margin of error lower than 9%. Zabrovskiy et al. (2021) achieve an error rate of equal to or less than 8%. Our proposal achieves the lowest error rate of less than 6.3%. With rates below 10%, these systems become more practical for functional use.

6. Conclusions and future work

In this paper a per-title and per-segment video encoding DNN-based has been proposed. A dataset has been generated to train the DNN that, once trained, gives the CRF adapted to each segment to achieve a target quality VMAF. An analysis of the dependency of the features with the resolution has been performed and we have shown that features can be extracted from 240p resolution without information loss while reducing the computational time. Details of the dataset, the DNN architecture, and the training process have been provided. Finally, we have validated the proposed scheme encoding a set of videos, not used during the training phase, requesting four different VMAF targets. Results show that the system adapts the CRF to each segment and that the final videos have a mean deviation of 1.84% with respect to the requested VMAF.

In this work only the 1080p resolution has been taken into account, in future work we plan to generate a dataset for different resolutions

Table 7
Qualitative comparison among previous work and our proposal.

Comparison Issues	Related Work					
	Covell et al. (2016)	Sun et al. (2018)	Cheng and Zhang (2019)	Zabrovskiy et al. (2021)	Moina-Rivera, Gutiérrez-Aguado, Garcia-Pineda, and Claver (2021)	Our proposal
Is it a Quality-based solution?	No	No	No	No	Yes, it uses VMAF metric	Yes, it uses VMAF metric
Machine learning technique	Deep Neural Network	Shallow Neural Network	Shallow Neural Network	Deep Neural Network	Does not use machine learning techniques	Deep Neural Network
Training and validation video segments	14000	27372	5031	349578	–	23028
Testing video segments	1000	4787	12	87395	–	1840
Video Features	1. Motion vector (MV) per macroblock (MB) 2. Percentage of texture 3. Percentage of I frames 4. Percentage of skipped frames 5. Complexity of encoding 6. Quantization parameter 7. Video resolution 8. Frame rate 9. Logarithm of original bitrate	1. Average PSNR of each channel & only Y channel 2. Encoded Bitrate 3. Percentage of texture relative to total bits 4. Percentage of I MB & P frames 5. Average number of bits & for texture for each MBs 6. Percentage of I & P MB 7. Percentage of zero MV 8. Average MV for P and B frames 9. Average MV difference	1. A score of encoding cost 2. Sum of pixels for each channel 3. Square sum of pixels for each channel 4. A score of AC per MB 5. Percentage of I MBs in P frame 6. Length of the MV for each predicted MB	1. Heigh and width 2. Number of pixels 3. Encoding preset 4. Encoding bitrate 5. Spatial Information (SI) at low resolution (LR) 6. Temporal Information (TI) at LR 7. Input segment size 8. Segment size at LR 9. Segment duration 10. Frame rate	None	1. Luminance (Y) at LR 2. Chrominance values (U, V) at LR 3. Saturation value at LR 4. Value of HUE at LR 5. Normalized graylevel entropy (S(Y), S(U), S(V)) at LR 6. TI at LR 7. Frame rate 8. Scene detection
Does it use a downscaling segment to extract features?	No	No	NO	Yes, but only some of them	Yes	Yes
Passes of encoding/transcoding	1 or 2, two-passes for a better accuracy	2	1 or 2, two-passes for a better accuracy	1	1	1
Main idea of the proposal	A neural network to estimate the correct CRF value for a desired bitrate.	A resolution-free two-pass sequence-level rate control scheme to significantly improve the bitrate accuracy.	A GOP-level rate control scheme to explore how to relate the target bitrate with CRF directly in a one-pass encoding framework.	An approach to calculate optimized multi-resolution bitrate ladder quickly and accurately for each video scene.	A system capable of encoding a video in several resolutions given the desired value of an objective metric (VMAF).	An encoding scheme based on a DNN, to perform a per-title and per-segment adaptive estimation of the encoding parameter to achieve a target video quality (VMAF) of the encoded video.
Average error	<= 20%	10% – 20%	<= 20%	<= 8%	<9%	<6,3%

and train the network to output CRF and resolution for a target VMAF. Besides, we plan to explore other features and other codecs. Finally, we will explore end-to-end DNN architectures to avoid the manual feature extraction.

CRediT authorship contribution statement

Francisco Micó-Enguídanos: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Resources, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Wilmer Moina-Rivera:** Conceptualization, Data curation, Investigation, Resources, Software, Visualization, Writing – original draft, Writing – review & editing. **Juan Gutiérrez-Aguado:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Miguel Garcia-Pineda:** Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

Grant by PID2021-126209OB-I00 funded by MCIN/AEI/10.13039/501100011033 and by "ERDF A way of making Europe", by the "European Union".

References

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., et al. (2015). TensorFlow: Large-scale machine learning on heterogeneous systems. Software available from tensorflow.org.

Begen, A. C., & Syed, Y. (2018). Are the streamingformat wars over? In *2018 IEEE international conference on multimedia and expo workshops* (pp. 1–4). Los Alamitos, CA, USA: IEEE Computer Society.

Cheng, B., & Zhang, Y. (2019). A neural network approach to GOP-level rate control of x265 using lookahead. In *2019 picture coding symposium* (pp. 1–5).

Cheon, M., & Lee, J.-S. (2018). Subjective and objective quality assessment of compressed 4K UHD videos for immersive eperience. *IEEE Transactions on Circuits and Systems for Video Technology*, *28*(7), 1467–1480.

Chollet, F., et al. (2015). Keras. https://keras.io. (Accessed 25 July 2022).

Cock, J. D., Mavlankar, A., Moorthy, A., & Aaron, A. (2016). A large-scale video codec comparison of x264, x265 and libvpx for practical VOD applications. In A. G. Tescher (Ed.), *Applications of digital image processing XXXIX, vol. 9971* (pp. 363–379). International Society for Optics and Photonics, SPIE.

Covell, M., Arjovsky, M., Lin, Y.-c., & Kokaram, A. (2016). Optimizing transcoder quality targets using a neural network with an embedded bitrate model. *Electronic Imaging*, *2016*(2), 1–7.

FFmpeg Team (2022). FFmpeg: A complete, cross-platform solution to record, convert and stream audio and video. http://www.ffmpeg.org/. (Accessed 25 July 2022).

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., et al. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362.

ITU-T (1999). *Recommendation P.910, Subjective video quality assessment methods for multimedia applications*. ITU Telecommunication Standardization Sector, https://handle.itu.int/11.1002/1000/14828. (Accessed 25 July 2022).

Kah, A., Friedrich, C., Rusert, T., Burgmair, C., Ruppel, W., & Narroschke, M. (2021). Fundamental relationships between subjective quality, user acceptance, and the VMAF metric for a quality-based bit-rate ladder design for over-the-top video streaming services. In A. G. Tescher, & T. Ebrahimi (Eds.), *Applications of digital image processing XLIV, vol. 11842* (pp. 316–325). International Society for Optics and Photonics, SPIE.

Kemp, S. (2022). *Digital 2022: Global overview report*. We Are Social and Hootsuite (Accessed 25 July 2022),

Li, Z., Aaron, A., Katsavounidis, I., Moorthy, A., & Manohara, M. (2016). Toward a practical perceptual video quality metric. *The Netflix Tech Blog*, *6*(2).

Li, Z., Bampis, C., Novak, J., Aaron, A., Swanson, K., Moorthy, A., et al. (2018). VMAF: The journey continues. *Netflix Technology Blog*, *25*.

Lim, M. K., Tang, S., & Chan, C. S. (2014). iSurveillance: Intelligent framework for multiple events detection in surveillance videos. *Expert Systems with Applications*, *41*(10), 4704–4715.

Mercat, A., Viitanen, M., & Vanne, J. (2020). UVG dataset: 50/120fps 4K sequences for video codec analysis and development. In *Proceedings of the 11th ACM multimedia systems conference* (pp. 297–302). New York, NY, USA: Association for Computing Machinery.

Moina-Rivera, W., Gutiérrez-Aguado, J., & Garcia-Pineda, M. (2021). Multi-resolution quality-based video coding system for DASH scenarios. In *Proceedings of the 31st ACM workshop on network and operating systems support for digital audio and video* (pp. 42–49). New York, NY, USA: Association for Computing Machinery.

Moina-Rivera, W., Gutiérrez-Aguado, J., Garcia-Pineda, M., & Claver, J. M. (2021). Improving DASH encoding with scenes and downscaling techniques for VoD streaming. In *2021 IEEE global communications conference* (pp. 1–6).

Nandakumar, D., Wu, Y., Wei, H., & Ten-Ami, A. (2019). On the accuracy of video quality measurement techniques. In *2019 IEEE 21st international workshop on multimedia signal processing* (pp. 1–6).

O'Malley, T., Bursztein, E., Long, J., Chollet, F., Jin, H., Invernizzi, L., et al. (2019). KerasTuner. https://github.com/keras-team/keras-tuner. (Accessed 25 July 2022).

Rao, R. R. R., Göring, S., Robitza, W., Feiten, B., & Raake, A. (2019). AVT-VQDB-UHD-1: A large scale video quality database for UHD-1. In *2019 IEEE ISM* (pp. 1–8).

Rassool, R. (2017). VMAF reproducibility: Validating a perceptual practical video quality metric. In *2017 IEEE international symposium on broadband multimedia systems and broadcasting* (pp. 1–2).

Richardson, I. E. (2002). *Video codec design: Developing image and video compression systems*. John Wiley & Sons.

Robitza, W., & Krasula, L. (2022). SITI: Spatial information/temporal information. https://pypi.org/project/siti-tools/. (Accessed 25 July 2022).

Sabokrou, M., Pourreza, M., Li, X., Fathy, M., & Zhao, G. (2021). Deep-HR: Fast heart rate estimation from face video under realistic conditions. *Expert Systems with Applications*, *186*, Article 115596.

Salvador, A., Nogueira, J., & Sargento, S. (2015). QoE assessment of HTTP adaptive video streaming. In S. Mumtaz, J. Rodriguez, M. Katz, C. Wang, & A. Nascimento (Eds.), *Wireless internet* (pp. 235–242). Cham: Springer International Publishing.

Song, L., Tang, X., Zhang, W., Yang, X., & Xia, P. (2013). The SJTU 4K video sequence dataset. In *2013 fifth international workshop on quality of multimedia experience* (pp. 34–35).

Sun, Y., Jin, M., Li, L., & Li, Z. (2018). A machine learning approach to accurate sequence-level rate control scheme for video coding. In *2018 25th IEEE international conference on image processing* (pp. 1013–1017).

Vigier, T., Rousseau, J., Da Silva, M. P., & Le Callet, P. (2016). A new HD and UHD video eye tracking dataset. In *Proceedings of the 7th international conference on multimedia systems*. New York, NY, USA: Association for Computing Machinery.

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, *17*, 261–272.

Wang, Y., Inguva, S., & Adsumilli, B. (2019). YouTube UGC dataset for video compression research. In *2019 IEEE 21st international workshop on multimedia signal processing*. IEEE.

Zabrovskiy, A., Agrawal, P., Timmerer, C., & Prodan, R. (2021). FAUST: Fast per-scene encoding using entropy-based scene detection and machine learning. In *2021 30th conference of open innovations association FRUCT* (pp. 292–302).